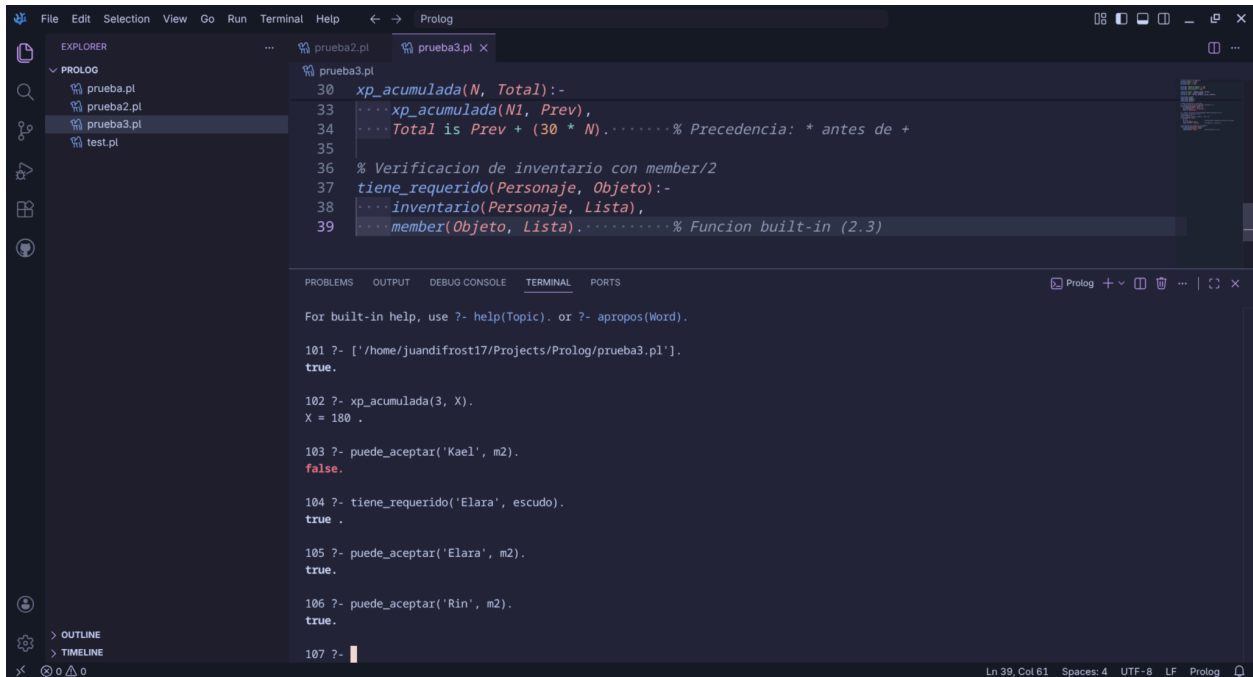


Fecha: 14/05/2026

Nombre: Juan Diego Sotomayor

Capturas - Prolog #3

1.



```
File Edit Selection View Go Run Terminal Help Prolog
EXPLORER
PROLOG
prueba.pl
prueba2.pl
prueba3.pl
test.pl
prueba3.pl
30 xp_acumulada(N, Total):-
33     xp_acumulada(N1, Prev),
34     Total is Prev + (30 * N).....% Precedencia: * antes de +
35
36 % Verificacion de inventario con member/2
37 tiene_requerido(Personaje, Objeto):-
38     inventario(Personaje, Lista),
39     member(Objeto, Lista).....% Funcion built-in (2.3)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
For built-in help, use ?- help(Topic). or ?- apropos(Word).

101 ?- ['/home/juandifrost17/Projects/Prolog/prueba3.pl'].
true.

102 ?- xp_acumulada(3, X).
X = 180 .

103 ?- puede_aceptar('Kael', m2).
false.

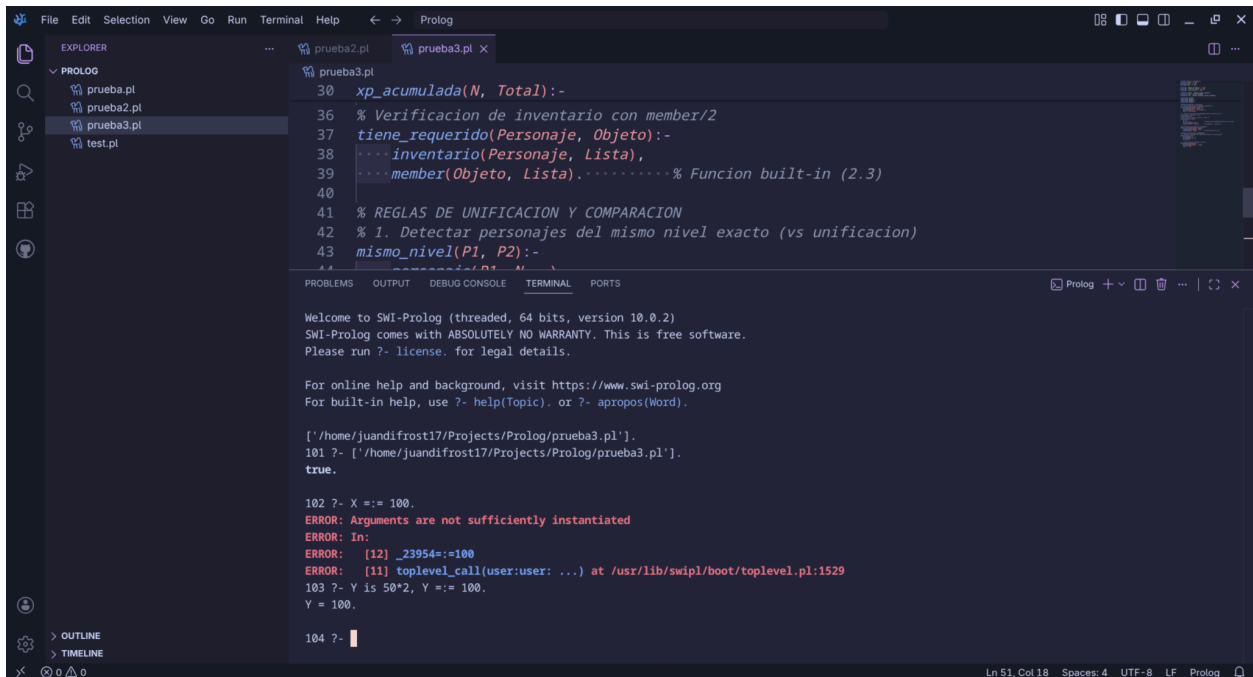
104 ?- tiene_requerido('Elara', escudo).
true .

105 ?- puede_aceptar('Elara', m2).
true.

106 ?- puede_aceptar('Rin', m2).
true.

107 ?-
```

2.



```
File Edit Selection View Go Run Terminal Help Prolog
EXPLORER
PROLOG
prueba.pl
prueba2.pl
prueba3.pl
test.pl
prueba3.pl
30 xp_acumulada(N, Total):-
36 % Verificacion de inventario con member/2
37 tiene_requerido(Personaje, Objeto):-
38     inventario(Personaje, Lista),
39     member(Objeto, Lista).....% Funcion built-in (2.3)
40
41 % REGLAS DE UNIFICACION Y COMPARACION
42 % 1. Detectar personajes del mismo nivel exacto (vs unificacion)
43 mismo_nivel(P1, P2):-
44     xp_acumulada(P1, M1),
45     xp_acumulada(P2, M2),
46     M1 = M2.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

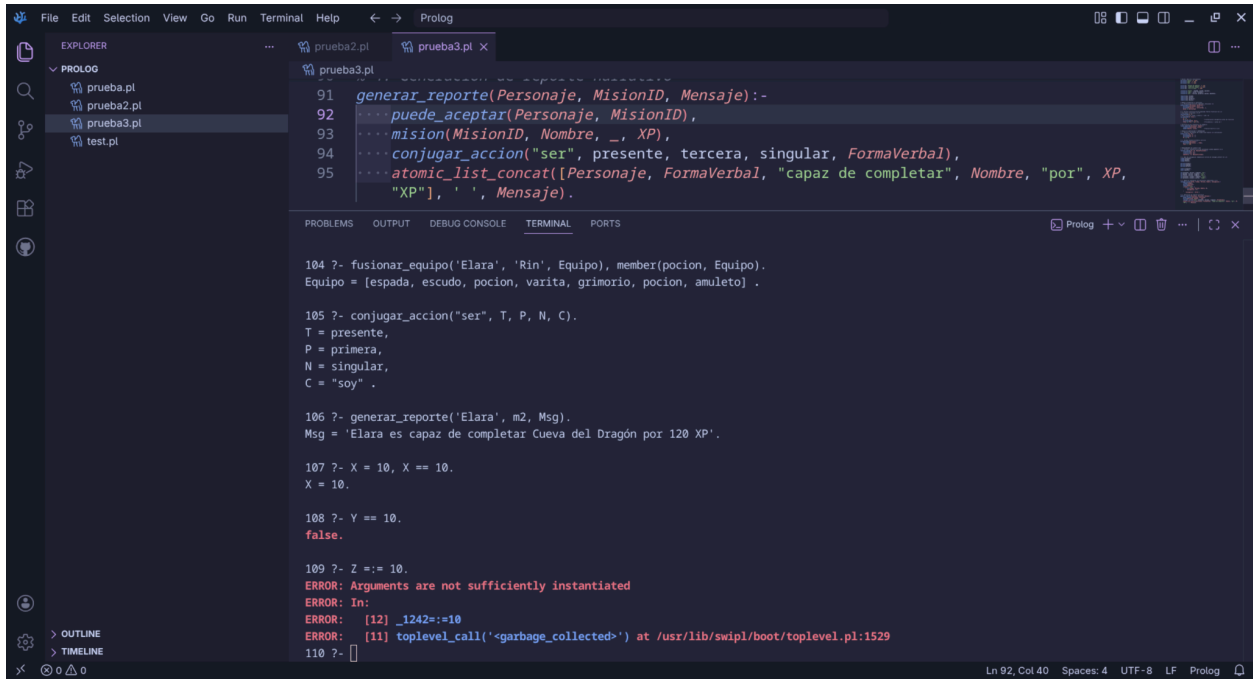
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

['/home/juandifrost17/Projects/Prolog/prueba3.pl'].
101 ?- ['/home/juandifrost17/Projects/Prolog/prueba3.pl'].
true.

102 ?- X =:= 100.
ERROR: Arguments are not sufficiently instantiated
ERROR: In:
ERROR: [12] _23954=:=100
ERROR: [11] toplevel_call(user:user: ...) at /usr/lib/swipl/boot/toplevel.pl:1529
103 ?- Y is 50*2, Y =:= 100.
Y = 100.

104 ?-
```

3.



The screenshot shows a Prolog IDE with a code editor and a terminal window. The code editor contains the following Prolog code:

```
101 generar_reporte(Personaje, MisionID, Mensaje):-
102     puede_aceptar(Personaje, MisionID),
103     mision(MisionID, Nombre, _, XP),
104     conjugar_accion("ser", presente, tercera, singular, FormaVerbal),
105     atomic_list_concat([Personaje, FormaVerbal, "capaz de completar", Nombre, "por", XP,
106                        "XP"], ' ', Mensaje).
```

The terminal window shows the following output:

```
104 ?- fusionar_equipo('Elara', 'Rin', Equipo), member(pocion, Equipo).
Equipo = [espada, escudo, pocion, varita, grimorio, pocion, amuleto] .

105 ?- conjugar_accion("ser", T, P, N, C).
T = presente,
P = primera,
N = singular,
C = "soy" .

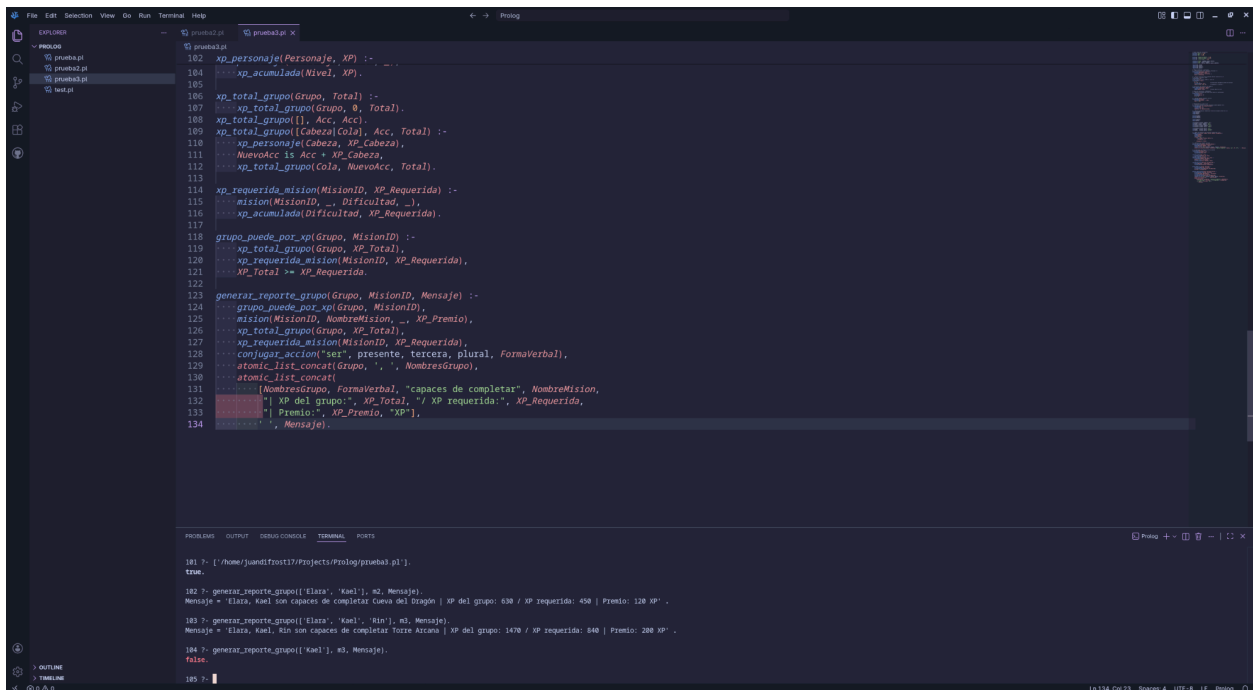
106 ?- generar_reporte('Elara', m2, Msg).
Msg = 'Elara es capaz de completar Cueva del Dragón por 120 XP' .

107 ?- X = 10, X == 10.
X = 10.

108 ?- Y == 10.
false.

109 ?- Z =:= 10.
ERROR: Arguments are not sufficiently instantiated
ERROR: In:
ERROR: [12] _1242:=:10
ERROR: [11] toplevel_call('<garbage_collected>') at /usr/lib/swipl/boot/toplevel.pl:1529
110 ?- []
```

4. Tarea (ENFOQUE PRINCIPAL)



The screenshot shows a Prolog IDE with a code editor and a terminal window. The code editor contains the following Prolog code:

```
102 xp_personaje(Personaje, XP) :-
103     xp_acumulada(Nivel, XP).
104 xp_acumulada(Nivel, XP) :-
105     xp_total_grupo(Grupe, Total) :-
106     xp_total_grupo(Grupe, 0, Total).
107 xp_total_grupo([_], Acc, Acc).
108 xp_total_grupo([Cabeza|Cola], Acc, Total) :-
109     xp_personaje(Cabeza, XP_Cabeza),
110     NuevoAcc is Acc + XP_Cabeza,
111     xp_total_grupo(Cola, NuevoAcc, Total).
112 xp_total_grupo(Cola, NuevoAcc, Total).
113 xp_requerida_mision(MisionID, XP_Requerida) :-
114     mision(MisionID, _, Dificultad, _),
115     xp_acumulada(Dificultad, XP_Requerida).
116 xp_acumulada(Dificultad, XP_Requerida).
117 grupo_puede_por_xp(Grupe, MisionID) :-
118     xp_total_grupo(Grupe, XP_Total),
119     xp_requerida_mision(MisionID, XP_Requerida),
120     XP_Total >= XP_Requerida.
121 xp_requerida_mision(MisionID, XP_Requerida).
122 generar_reporte_grupo(Grupe, MisionID, Mensaje) :-
123     grupo_puede_por_xp(Grupe, MisionID),
124     mision(MisionID, NombreMision, _, XP_Premio),
125     xp_total_grupo(Grupe, XP_Total),
126     xp_requerida_mision(MisionID, XP_Requerida),
127     conjugar_accion("ser", presente, tercera, plural, FormaVerbal),
128     atomic_list_concat(Grupe, ' ', NombresGrupo),
129     atomic_list_concat([NombresGrupo, FormaVerbal, "capaces de completar", NombreMision,
130                        "XP del grupo:", XP_Total, " / XP requerida:", XP_Requerida,
131                        " Premio:", XP_Premio, "XP"],
132                        Mensaje).
```

The terminal window shows the following output:

```
101 ?- ['(home/juandifur17/Projects/Prolog/prueba3.pl)'].
true.

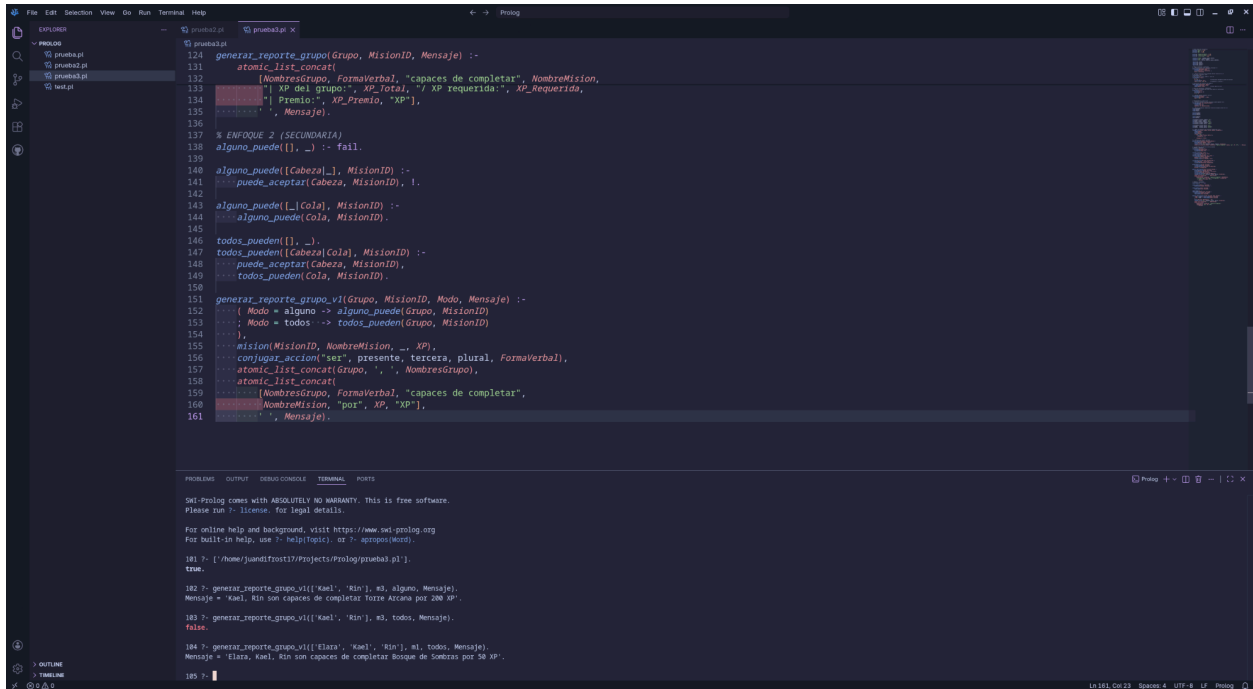
102 ?- generar_reporte_grupo('Elara', 'Kael', m2, Mensaje).
Mensaje = 'Elara, Kael son capaces de completar Cueva del Dragón | XP del grupo: 630 / XP requerida: 450 | Premio: 120 XP' .

103 ?- generar_reporte_grupo('Elara', 'Kael', 'Rin', m3, Mensaje).
Mensaje = 'Elara, Kael, Rin son capaces de completar Torre Arcana | XP del grupo: 1470 / XP requerida: 840 | Premio: 280 XP' .

104 ?- generar_reporte_grupo('Kael', m3, Mensaje).
false.

105 ?- []
```

5. Tarea (ENFOQUE SECUNDARIO)



```
121 generar_reporte_grupo(Grupo, MisionID, Mensaje) :-
122     atomic_list_concat(
123         [NombresGrupo, FormaVerbal, "capaces de completar", NombreMision,
124           "XP del grupo:", XP_Total, "/ XP requerida:", XP_Requerida,
125           "Frente", XP_Promio, "XP"],
126         Cabecera, Mensaje).
127
128 % ENFOQUE 2 (SECUNDARIO)
129 alguno_puede([], _) :- fail.
130
131 alguno_puede([Cabeza], MisionID) :-
132     puede_aceptar(Cabeza, MisionID), !.
133
134 alguno_puede([_|Cola], MisionID) :-
135     alguno_puede(Cola, MisionID).
136
137 todos_pueden([], _) :- fail.
138 todos_pueden([Cabeza|Cola], MisionID) :-
139     puede_aceptar(Cabeza, MisionID), !,
140     todos_pueden(Cola, MisionID).
141
142 generar_reporte_grupo_v1(Grupo, MisionID, Modo, Mensaje) :-
143     (Modo = alguno => alguno_puede(Grupo, MisionID)
144     ; Modo = todos => todos_pueden(Grupo, MisionID)
145     ; true),
146     mision(MisionID, NombreMision, _, XP),
147     conjugate_accion(Ver, presente, tercera, plural, FormaVerbal),
148     atomic_list_concat(Grupo, ', ', NombresGrupo),
149     atomic_list_concat(
150         [NombresGrupo, FormaVerbal, "capaces de completar",
151          NombreMision, "por", XP, "XP"],
152         Cabecera, Mensaje).
153
154 181 ?- ["/home/juanitofrost/Projects/Prolog/psuab3.pl].
155 true.
156
157 182 ?- generar_reporte_grupo_v1(['Kael', 'Rin'], m3, alguno, Mensaje).
158 Mensaje = 'Kael, Rin son capaces de completar Torre Arcana por 200 XP'.
159
160 183 ?- generar_reporte_grupo_v1(['Kael', 'Rin'], m3, todos, Mensaje).
161 false.
162
163 184 ?- generar_reporte_grupo_v1(['Elara', 'Kael', 'Rin'], m1, todos, Mensaje).
164 Mensaje = 'Elara, Kael, Rin son capaces de completar Bosque de Sombras por 50 XP'.
165 ?- |
```